



Reinforcement Learning for Mid-to-Mid Motion Planning in Autonomous Driving

Islem Kobbi, Hussam Atoui, Fawzi Nashashibi

► To cite this version:

Islem Kobbi, Hussam Atoui, Fawzi Nashashibi. Reinforcement Learning for Mid-to-Mid Motion Planning in Autonomous Driving. ITSC 2025 - The IEEE International Conference on Intelligent Transportation Systems, Nov 2025, Gold Coast, Australia. <hal-05294346>

HAL Id: hal-05294346

<https://hal.science/hal-05294346v1>

Submitted on 2 Oct 2025

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



HAL Authorization

Reinforcement Learning for Mid-to-Mid Motion Planning in Autonomous Driving

Islem Kobbi¹, Hussam Atoui^{1,2} and Fawzi Nashashibi¹

Abstract—This paper proposes a reinforcement learning-based mid-to-mid motion planning framework for autonomous vehicles, where an agent receives structured mid-level observations and outputs continuous paths suitable for tracking. To accelerate training and avoid the inefficient early phase of RL, the agent is first initialized using imitation learning (IL) via behavior cloning (BC) from a rule-based motion planner (RBMP). RL training then takes over using Proximal Policy Optimization (PPO), where the agent learns directly from interaction to achieve high-performance driving behavior. This mid-to-mid architecture preserves the transparency and modularity of classical decision-making pipelines while leveraging the full capabilities of RL. Experimental results show that the RL-trained agent not only surpasses the initial IL-based policy by a wide margin but also clearly outperforms the RBMP baseline, demonstrating that RL is solely responsible for achieving superior, robust, and adaptable motion planning.

Keywords— Reinforcement Learning, Motion Planning, Autonomous Driving, Imitation Learning, Behavior Cloning, Proximal Policy Optimization

I. INTRODUCTION

Motion planning is a key component of autonomous driving systems, acting as the bridge between high-level navigation goals and low-level vehicle control [1]. Traditional systems follow a modular pipeline consisting of four main stages: *perception*, *route planning*, *motion planning*, and *high-level control*. This classical structure, as illustrated in Figure 1, offers transparency, interpretability, and ease of debugging and validation.

Each of these modules is typically implemented using a rule-based logic or utility functions. While such approaches provide deterministic and explainable behavior, they rely on fixed heuristics and often lack adaptability in dynamic or unforeseen scenarios. Given the complexity and unpredictability of real-world driving environments, these limitations negatively impact the robustness and efficiency of autonomous driving systems. Recent advancements in artificial intelligence, particularly deep learning and reinforcement learning, hold great promise for replacing individual modules with models that learn directly from data or interact with the environment.

To apply AI-based solutions, two common strategies have emerged. One is the *end-to-end* approach, which aims to

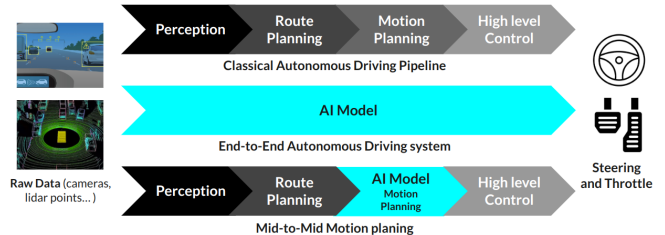


Fig. 1. Comparison between classical multi-stage, end-to-end, and mid-to-mid decision-making pipelines for autonomous vehicles.

address the limitations of classical pipelines by using a single deep learning model to map perception inputs, or even raw sensor data, directly to control commands [2]. This design reduces error propagation between modules and simplifies the training process. However, it sacrifices modularity, interpretability, and safety assurance, properties that are essential in safety-critical domains such as autonomous driving.

To balance adaptability with system transparency, we adopt a **mid-to-mid architecture**, shown in the third row of Figure 1. In this setup, only the motion planning module is replaced with a learning-based agent, while the upstream perception and downstream control modules remain separated. This approach maintains the modularity and explainability of the overall system while enabling adaptive, data-driven motion planning.

Traditional motion planners, whether rule-based [3] or utility-based [4], often underperform in highly dynamic or uncertain driving conditions due to their limited flexibility and reliance on pre-programmed behavior. In contrast, reinforcement learning (RL) allows agents to learn optimal policies through direct interaction with the environment. RL can reason about long-term consequences, adapt to previously unseen states, and generate smooth, safe, and efficient trajectories [5].

Various RL algorithms have been investigated for autonomous driving, including Deep Q-Networks (DQN)[5], Deep Deterministic Policy Gradient (DDPG)[6], and Soft Actor-Critic (SAC)[7], each offering distinct trade-offs in terms of sample efficiency, training stability, and suitability for continuous control. Among these, Proximal Policy Optimization (PPO)[8] is particularly well-suited to motion planning due to its stability and robustness in high-dimensional, continuous action spaces.

Despite its advantages, RL typically requires significant exploration and long training times, especially in tasks with sparse rewards or complex objectives. To accelerate

¹I. Kobbi, H. Atoui and F. Nashashibi are with the ASTRA Team, National Institute for Research in Digital Science and Technology (INRIA), 75012 Paris, France. islem.kobbi@inria.fr, hussam.atoui@inria.fr, fawzi.nashashibi@inria.fr

²H. Atoui is with Valeo in Créteil, France. hussam.atoui@valeo.com

convergence and guide early learning, the RL agent can be initialized using **Imitation Learning (IL)** [9], [10], specifically Behavior Cloning (BC) [11], trained on expert demonstrations. IL provides a good initial policy by mimicking expert behavior, reducing the burden of early-stage exploration. Importantly, all further improvements and generalization capabilities are achieved through reinforcement learning.

This work introduces an RL-based mid-to-mid motion planner for autonomous driving, with the following key contributions:

- An RL-based Motion Planner (RLMP) is designed where it receives structured mid-level observations and generates continuous, smooth paths using Hermite splines.
- Implementation and training in a customized MetaDrive simulator, with BC for initialization and PPO for policy optimization.
- Extensive evaluation against a rule-based baseline, demonstrating clear improvements in efficiency, smoothness, and overall performance.

II. RL-BASED MOTION PLANNER DESIGN

As illustrated in Figure 1, the proposed system retains all traditional decision-making modules to maintain transparency and modularity. The perception module converts raw sensor inputs into structured observations, the route planner provides fixed local goals, and the control module computes high-level commands to track the planned trajectories. In simulation, perception is error-free, goals remain static, and a finely tuned controller minimizes tracking errors. This setup ensures the agent remains fully responsible for its decisions during training, as the reward function is computed primarily based on the planned trajectory rather than the actual vehicle execution.

A. Observation Space

Representing the environment observation space for RL agents can be approached in several ways. Some methods employ processed representations like bird's eye view maps or occupancy grids [12]. Recent approaches have also incorporated attention mechanisms to handle variable input sizes [13]. In this work, we adopt a simple fixed-size vector representation that offers a computational efficiency.

The observation vector has a fixed size of 91×1 and is composed of the following components:

- **Ego State:** $Speed_{ego}, Width_{ego}, Length_{ego}$
- **Surrounding Vehicles Information:** $[(x, y)_{veh_i}, Speed_{veh_i}, Head_{veh_i}, Width_{veh_i}, Length_{veh_i}] \times 8$ vehicles, as shown in Figure 2, positions are categorized as front, front-left, front-right, right, left, rear, rear-left, and rear-right.
- **Lane Information:** $[lane_width, lane_type \text{ (normal or emergency lane)}, (x, y) \text{ positions for 4 equidistant points along each lane}]$, where the distance between points is relative to the ego vehicle's speed.

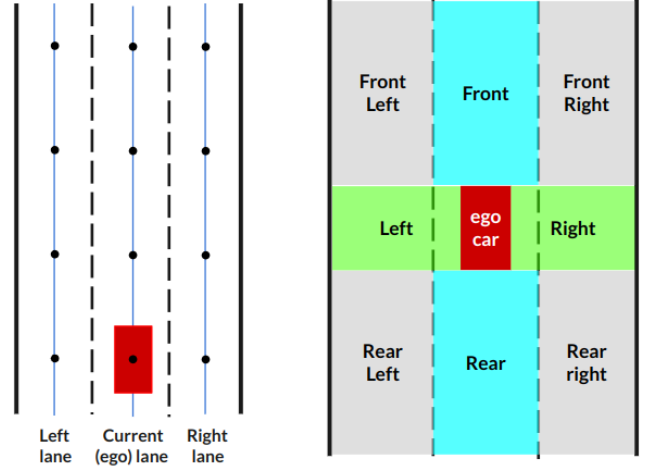


Fig. 2. Illustration of the observation space. Left: Points describing the lane geometry. Right: Eight surrounding zones used to encode nearby vehicle information.

- **Distance to Goal:** $(x, y)_{Goal}$ the relative distance to the end of the current lane
- **Previous Action:** The agent's last action, included to provide temporal context

All positional and speed data are expressed relative to the ego vehicle's frame. Moreover, all features are normalized within the range $[-1, 1]$ for stable learning.

As shown in Figure 2, lane geometry is represented by four equidistant points per lane, which is sufficient to accurately describe the road shape in our straight highway environment. If multiple vehicles occupy the same surrounding zone, only the closest vehicle is considered. If no vehicle is present in a given zone, the corresponding entries are filled with zeros. Similarly, if there are no adjacent lanes to the left or right, the related lane fields are zeroed out.

B. Action Space

Since the agent represents a path planner, its actions must generate continuous trajectories. A straightforward approach would be to directly output a sequence of waypoints; however, this would result in a large action space, increasing the complexity of the model and the learning process. Instead, a more compact representation is used based on Cubic Hermite splines.

The agent outputs parameters for two control points, while the first point is fixed at the vehicle's current pose. As shown in Figure 3, each control point is described by an (x, y) position, a heading angle θ , and a reference speed V , resulting in an output vector composed of eight values. Cubic Hermite splines offer a good balance between flexibility and interpretability, as the outputs represent meaningful control points rather than abstract function parameters, making them preferable to alternatives such as Bézier curves [14] or sigmoid-based functions [15] in this context.

The agent's action output is defined as:

$$\text{action: } [X_1, Y_1, \theta_1, V_1, X_2, Y_2, \theta_2, V_2] \quad (1)$$

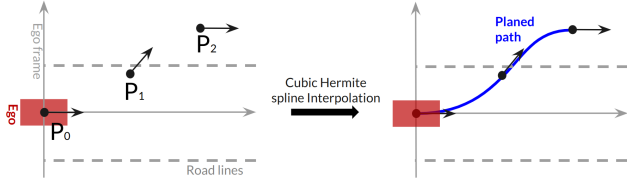


Fig. 3. Illustration of the agent's action space: control points P_1 and P_2 , and their conversion into a continuous trajectory.

where the control points are:

$$\begin{aligned} P_0 \text{ (current ego pose)} & (0, 0, 0, V_0) \\ P_1 \text{ (first control point)} & (X_1, Y_1, \theta_1, V_1) \\ P_2 \text{ (second control point)} & (X_2, Y_2, \theta_2, V_2) \end{aligned}$$

The (x, y) positions are expressed relative to the frame of the preceding control point: P_1 is relative to P_0 , and P_2 is relative to P_1 . In contrast, the heading angles θ and speeds V are expressed in absolute terms.

This relative positioning strategy constrains the lateral displacement between consecutive control points, while still offering a wide range of motion planning flexibility. The longitudinal position of the control points scales linearly with the vehicle's current speed, ranging from a maximum of 20 meters when stationary to 50 meters at the maximum speed of 25 m/s. Lateral displacements are constrained within $[-4, 4]$ meters for each control point, resulting in a full lateral planning range of 8 meters. The heading angles are limited to the range $[-\pi/6, \pi/6]$. Finally, all output values are normalized within the range $[-1, 1]$ to facilitate stable neural network training.

C. Reward Function

The reward function is carefully designed to reflect the task objectives while maintaining computational efficiency, enabling the value function to learn the reward structure effectively. The total reward is divided into two main components: a *Path Reward* computed prior to execution, and an *Execution Reward* computed during path tracking.

1) *Path Reward*: R_{path} is the primary component, evaluated solely based on the planned trajectory before it is executed. It comprises the following terms:

- **Progress Reward**: Encourages high reference speeds and forward motion.

$$R_{prog} = C_{speed} \times \min\left(\frac{V_1 + V_2}{2V_{idle}}, 1\right) + C_{pos} \times \frac{X_1 + X_2}{2X_{max_dist}} \quad (2)$$

where C_{speed} and C_{pos} are weighting constants; V_1 , V_2 , X_1 , and X_2 represent the speed references and longitudinal positions of the two control points,

respectively. V_{idle} denotes the ideal cruising speed (80km/h), and X_{max_dist} is the maximum possible planning distance. The *min* operations normalize the reward within $[0, 1]$.

- **Adherence to Traffic Rules**: Rewards lane-centering behavior and compliance with speed limits. Lane-centering reward is computed by selecting n equidistant points along the planned trajectory and calculating their distance to the nearest lane centerline:

$$R_{cl} = 1 - \frac{1}{2n \times lane_width} \sum_{i=1}^n \min(d_{i1}, d_{i2}, d_{i3}) \quad (3)$$

where d_{il} denotes the distance from point i to the center of lane l .

Exceeding the speed limit is penalized by:

$$R_{sl} = \min(V_{idle} - V, 0) \quad (4)$$

where V and V_{idle} are the current speed and the speed limit, respectively.

The final traffic adherence reward is then:

$$R_{atr} = C_{cl}R_{cl} + C_{sl}R_{sl} \quad (5)$$

where C_{cl} and C_{sl} are weighting constants.

- **Path Smoothness Reward**: Penalizes sharp lateral movements and abrupt heading changes, emphasizing trajectory smoothness. Heading angles are weighted three times more heavily than lateral displacements, as their effect on the curvature of the path is greater. Additionally, maintaining the ego vehicle's heading parallel to the road is prioritized, while greater tolerance is allowed for lateral displacements.

$$R_{sm} = C_{sm} \left(1 - \frac{|Y_1| + |Y_2| + 3(|\theta_1| + |\theta_2|)}{8} \right) \quad (6)$$

where C_{sm} is a constant, and Y_1 , Y_2 , θ_1 , and θ_2 are the lateral positions and heading angles of the two control points, respectively.

2) *Execution Reward*: R_{exe} is computed during the trajectory execution phase, evaluating potential collisions and off-road violations. It is calculated at each sub-step and accumulated, then added to the final step reward. For clarity, steps refer to the agent's decisions (providing a new planned path), while sub-steps correspond to the tracking of that path before requesting a new action from the agent.

The execution reward penalizes close proximity to other vehicles, collisions and going off-road:

$$R_{exe} = \begin{cases} 0 & \text{if } d_{lat} > 3 \text{ m and } d_{long} > 10 \text{ m} \\ \frac{-C_{dtc}}{d_{lat}} & \text{if } d_{lat} < 3 \text{ m and } d_{long} < h_{car} \\ \frac{-2C_{dtc}}{d_{long}} & \text{if } d_{long} < 10 \text{ m and } d_{long} < w_{car} \\ -C_{col} & \text{if a collision occurs} \\ -C_{out} & \text{if the vehicle leaves the road} \end{cases} \quad (7)$$

where d_{lat} and d_{long} are the minimum lateral and longitudinal distances to nearby vehicles, respectively. h_{car} and w_{car} denote the height and width of the ego vehicle. C_{dtc} , C_{col} and C_{out} are constant coefficients. Note that the agent is more tolerant of lateral proximity to adjacent vehicles than longitudinal proximity to vehicles in front. also in both cases (collision or off-road), the episode is immediately terminated.

The total step reward is thus:

$$R_{step} = R_{path} + \sum_{i=1}^{n_sub_steps} R_{exe}^i \quad (8)$$

where:

$$R_{path} = C_{atr}R_{atr} + C_{sm}R_{sm} + C_{prog}R_{prog} \quad (9)$$

$$R_{exe}^i = R_{out}^i + R_{col}^i \quad (10)$$

with C_{atr} , C_{sm} , and C_{prog} as weighting constants, and n_sub_steps representing the number of sub-steps taken while tracking the given path.

D. RL Algorithm

The agent is trained using the PPO algorithm [8], a widely adopted on-policy reinforcement learning method known for its stability and efficiency.

The policy network (actor) is implemented as a multi-layer perceptron (MLP) with hidden layer sizes of [256, 256, 128, 64, 32], and employs LeakyReLU activation functions. This architecture has been first validated through pretraining with BC, which demonstrated sufficient model capacity for the task.

The value network (critic) shares the same architectural configuration and activation functions as the actor.

III. TRAINING PROCESS

To simulate a highway driving environment, a customized version of the MetaDrive simulator is used. This version is adapted to provide the specific observation and action interfaces required by the learning agent. MetaDrive is selected for its balance between complexity and realism. By default, it accepts only low-level acceleration and steering commands.

To bridge the gap between the agent's high-level planning and the simulator's low-level control interface, an action interface is introduced. as shown in Figure 4, the agent outputs parameters defining control points, which the action interface uses to construct a continuous trajectory using cubic Hermite splines. This trajectory is then tracked by an integrated controller that combines Pure Pursuit [16] for lateral (steering) control and a PID controller for longitudinal (speed) control. These controllers compute the precise high-level control signals (steering angle and throttle) that are passed to the simulator.

Once a trajectory is generated, the controller tracks it over a predefined number of sub-steps before requesting a new action from the agent.

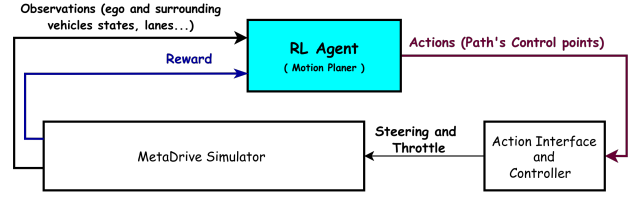


Fig. 4. System Architecture Overview: interaction flow between the agent, action interface, controller and the MetaDrive simulator

To speed up training and place the agent in a sufficiently productive training state, the policy is pre-trained using behavior cloning (BC). This allows the agent to acquire fundamental driving behaviors and bypasses the time-consuming initial exploration phase usually required in reinforcement learning.

A. BC Pre-trained

A classical Rule-Based Motion Planner (RBMP) has been used as the expert policy to generate a dataset of approximately 20,000 transitions, covering around 150 driving episodes. This dataset is then utilized to pretrain the actor (policy network) of the PPO agent via BC. This pretraining phase provided the agent with a structured initial policy, allowing it to exhibit basic driving behaviors and accelerating subsequent RL convergence.

B. RLMP Training

The training of the RL-based Motion Planner (RLMP) is conducted using PPO. The selected hyperparameters and reward constants are summarized in Table I. All other parameters are kept at their default values as defined by the StableBaselines3 (SB3) and MetaDrive frameworks.

Initially, the value function loss coefficient (v_{coef}) was set to 1.0 to accelerate the convergence of the value network, which had not been pretrained. Once the value loss fell below a threshold of 10, v_{coef} was reduced to its default value of 0.5, ensuring a balanced update between policy learning and value estimation.

IV. RESULTS

A. Training Results

Figure 5 shows the evolution of the cumulative reward and episode lengths over training steps. The agent's initial performance was significantly improved through BC initialization. Unlike standard RL training, where rewards typically start near zero or even negative, the BC-pretrained agent exhibited sufficiently good initial performance, avoiding common early-stage exploration pitfalls.

B. Validation Results

To assess performance, the RL-Based Motion Planner (RLMP) policy is evaluated against a Rule-Based Motion Planner (RBMP), which serves as the baseline, and the initial BC-Based Motion Planner (BCMP) policy trained using BC. Evaluation is conducted over 20 randomly generated test

TABLE I

GROUPED PPO TRAINING HYPERPARAMETERS AND ENVIRONMENT
REWARD FUNCTION COEFFICIENTS.

Parameter	Value
Value loss coefficient (v_{coef})	1 (init), then 0.5
Policy learning rate	$10^{-4} \rightarrow 10^{-6}$
Discount factor (γ)	0.95
GAE lambda (λ)	0.95
Buffer size	10^6
Entropy coefficient	0.01
Batch size	512
Epochs per update	10
PPO	
C_{speed}	0.7
C_{pos}	0.3
V_{idle}	22 m/s (80 km/h)
C_{cl}	0.5
C_{sl}	0.5
C_{atr}	0.3
C_{sm}	0.1
C_{prog}	0.6
C_{dtc}	0.2
C_{col}	5
C_{out}	5
Reward	

TABLE II

EVALUATION RESULTS OVER 20 TEST SCENARIOS.

Model	Cumulative Reward		Collisions	Off-Road
	Mean	Median		
BCMP (BC)	7.37	1.12	5	15
RLMP (PPO)	188.17	201.19	0	0
RBMP (Baseline)	180.39	176.29	0	0

performance could be obtained using more advanced IL methods [17], [18].

A qualitative comparison between the trajectories generated by RBMP and RLMP is shown in Figure 6. While the selected maneuvers (such as lane changes and car following) are often similar, execution differs significantly. On straight roads, RLMP tends to generate longer trajectories and maintain higher speeds. During overtaking, it follows smoother, less curved paths while remaining closer to the lane centerline, an effect encouraged by the reward structure, which promotes both smoothness and lane adherence.

V. DISCUSSION

The results demonstrate that the RLMP agent outperforms the baseline RBMP, confirming that RL can surpass expert level performance when properly initialized and guided. IL (specifically BC) is used as a shortcut to address the inefficiencies and time-consuming nature of early stage RL training. It provides fundamental driving behaviors and allows the agent to bypass the slow initial exploration phase, starting from a more productive training state.

The performance gains arise from RLMP's capacity to learn directly through interaction with the environment. While BC offers a strong initial policy, the BCMP alone remains unstable during deployment, frequently encountering out-of-distribution states that lead to collisions or off-road driving. The RL phase enables the agent to generalize to these unseen situations, improving stability and discovering more efficient trajectories. As a result, the agent not only matches but exceeds the performance of the RBMP, which originally served as the expert for BC training. High-level maneuvers such as lane keeping and lane changing remain aligned with RBMP strategies, demonstrating BC's role in transferring structured behavior. However, RL contributes further by optimizing trajectory quality, achieving higher efficiency, and producing smoother lane transitions.

Notably, these results are achieved using a standard PPO algorithm and a fixed-size observation space, without advanced architectures or task-specific enhancements. Future work could explore richer observation processing (such as attention mechanisms or transformer-based models) and more sophisticated RL algorithms. Additionally, increasing the quality and diversity of demonstration data, particularly through real-world human driving, could further strengthen behavioral priors and lead to even greater performance gains.

VI. CONCLUSIONS

This work introduces an RL-based mid-to-mid motion planning framework that preserves the modular structure of

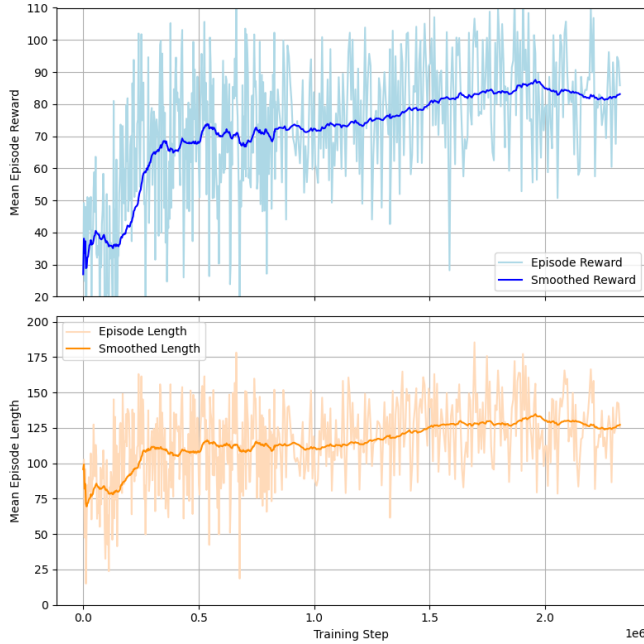


Fig. 5. Training progression under PPO: mean episode reward and episode length over training steps.

scenarios, with each agent tested on the same set of episodes to ensure a fair comparison. Performance is measured using the same reward function employed during training. To better capture long-term behavior, episode durations are extended, allowing agents to demonstrate sustained performance under varying traffic and road conditions.

As shown in Figure 7 and Table II, the RLMP outperforms the RBMP baseline, achieving the highest mean and median cumulative rewards. It also records zero off-road violations and collisions across all test cases. Note that the BC-initialized BCMP is included only to illustrate policy performance before and after the RL training. Higher

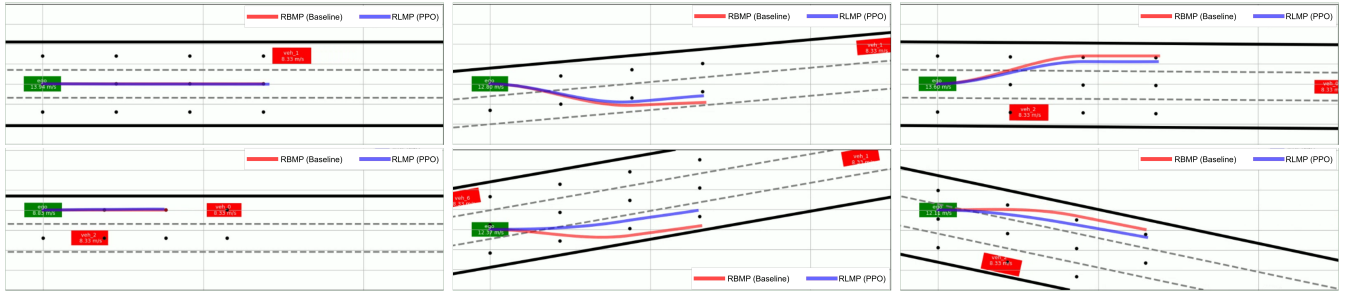


Fig. 6. Direct comparison between the trajectories generated by the RBMP (red) and the RLMP (blue) across various scenarios: straight driving, car following, and lane changes to the left and right.

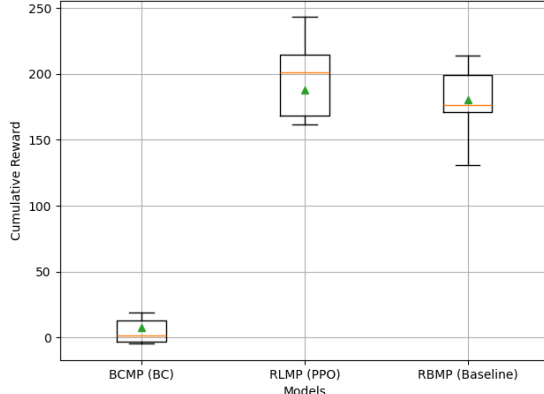


Fig. 7. Cumulative reward distributions for the evaluated models.

classical autonomous driving systems. The system processes structured observations and generates continuous trajectories using control points and cubic Hermite splines, providing both flexibility and interpretability.

By leveraging RL's capacity to learn from interaction, the agent achieves smoother, faster, and safer motion compared to the RBMP baseline. Future work may explore richer observation encodings, training with real-world demonstration data, and more advanced RL algorithms. Additionally, modifying the reward function could allow tuning of the agent's behavior, enabling driving styles that range from efficiency-oriented to more conservative or safety-focused.

Overall, the results demonstrate RL's potential to enhance the performance, safety, and adaptability of autonomous motion planning systems.

REFERENCES

- [1] F. J. G. Carpio and P. Resende, "Review of decision-making and planning approaches in automated driving," *IEEE Access*, vol. 10, pp. 100 348–100 366, 2022.
- [2] G. Lan and Q. Hao, "End-to-end planning of autonomous driving in industry and academia: 2022–2023," *arXiv preprint arXiv:2310.08730*, 2023.
- [3] F. Bouchard, S. Sedwards, and K. Czarnecki, "A rule-based behaviour planner for autonomous driving," in *Rules and Reasoning*. Springer, 2022, pp. 263–279.
- [4] Y. Kim, D.-S. Pae, S.-H. Jang, S.-W. Kang, and M.-T. Lim, "Reinforcement learning for autonomous vehicle using mpc in highway situation," in *Proc. 2022 Int. Conf. on Electronics, Information, and Communication (ICEIC)*, 2022, pp. 1–4.
- [5] A. M. G. Ruiz, H. Atoui, and O. Sename, "Design and experimental validation of rl-based decision-making system for autonomous vehicles," in *Proc. of the 11th International Conference on Mechatronics and Control Engineering*, 2024, pp. 99–116.
- [6] S. Wang, D. Jia, and X. Weng, "Deep reinforcement learning for autonomous driving," *arXiv preprint arXiv:1905.00944*, 2019.
- [7] M. Gao and D. E. Chang, "Autonomous driving based on modified sac algorithm through imitation learning pretraining," in *2021 21st International Conference on Control, Automation and Systems (ICCAS)*, 2021, pp. 1360–1364.
- [8] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [9] Z. Xie, Z. Lin, J. Li, S. Li, and D. Ye, "Pretraining in deep reinforcement learning: A survey," *arXiv preprint arXiv:2211.12486*, 2022.
- [10] I. Kobbi, A. Benamirouche, and M. Tadjine, "Enhancing pose estimation for mobile robots: A comparative analysis of deep reinforcement learning algorithms for adaptive extended kalman filter-based estimation," *Engineering Applications of Artificial Intelligence*, vol. 150, p. 110548, 2025.
- [11] F. Torabi, G. Warnell, and P. Stone, "Behavioral cloning from observation," *arXiv preprint arXiv:1805.01954*, 2018.
- [12] E. Leurent, "A survey of state-action representations for autonomous driving," *arXiv preprint arXiv:1811.10454*, 2018.
- [13] E. Leurent and J. Mercat, "Social attention for autonomous decision-making in dense traffic," *arXiv preprint arXiv:1911.12250*, 2019.
- [14] J.-W. Choi, R. Curry, and G. Elkaim, "Path planning based on bézier curve for autonomous ground vehicles," in *World Congress on Engineering and Computer Science*, 2008, pp. 158–166.
- [15] X. Huang, W. Zhang, and P. Li, "A path planning method for vehicle overtaking maneuver using sigmoid functions," *IFAC-PapersOnLine*, vol. 52, no. 8, pp. 422–427, 2019.
- [16] S. Macenski, S. Singh, F. Martin, and J. Gines, "Regulated pure pursuit for robot path tracking," *arXiv preprint arXiv:2302.07225*, 2023.
- [17] J. Ho and S. Ermon, "Generative adversarial imitation learning," *arXiv preprint arXiv:1606.03476*, 2016.
- [18] S. Ross, G. J. Gordon, and J. A. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," *arXiv preprint arXiv:1011.0686*, 2011.